เรียนรู้โมเดล ให้ใช้กับข้อมูลใหม่ได้

เข้าใจ Machine Learning, Bias–Variance และ Linear Regression ผ่านคำถาม การทดลอง และสถานการณ์ทำงาน

ฉบับอ่านและพิมพ์ • เปิด study_guide.html เพื่อใช้กราฟและตรวจคำตอบแบบโต้ตอบ
มีเฉลยแบบฝึกหัดท้ายเล่ม • ตัวอย่างตัวเลขและกราฟเป็นข้อมูลจำลอง

เริ่มต้น • แผนการเรียน

จากข้อมูลเก่า สู่คำทำนายที่ใช้ได้กับข้อมูลใหม่

คำถามหลักของสองบทนี้คือ “โมเดลเรียนรู้อะไร และเรารู้ได้อย่างไรว่ามันใช้กับงานจริงได้?”

กำหนดงาน	เรียนรู้จากข้อมูล	ทดสอบให้ตรงงาน
จะช่วยใครตัดสินใจอะไร?	เลือกข้อมูลและฝึกโมเดล	ตรวจข้อผิดพลาดและข้อจำกัด

เมื่ออ่านจบ ควรทำอะไรได้?

บท 07 • แนะนำ Machine Learning

1. อธิบาย ML และความสัมพันธ์กับ AI
2. แยก Supervised, Unsupervised และ Reinforcement Learning
3. อธิบาย workflow ตั้งแต่เตรียมข้อมูลถึงประเมินผล
4. อธิบาย Bias–Variance Tradeoff
5. ใช้ `fit / predict / score` เบื้องต้น
6. อธิบายบทบาทของ LLM ในงาน Data Science

บท 08 • Regression

1. อธิบายหลักการ Linear Regression
2. สร้าง Simple และ Multiple Linear Regression
3. คำนวณและตีความ MAE, MSE, RMSE และ R^2
4. อ่าน Residual Plots เพื่อตรวจข้อสมมติ
5. ปรับโมเดลด้วย Feature Selection และ Regularization

รายการนี้ย่อภาษาจากวัตถุประสงค์บนเว็บบท 07–08 ไม่ใช่การกำหนด CLO ใหม่ ข้อความต้นฉบับและแหล่งอ้างอิงอยู่ใน learning_outcomes.json และ source_audit.md

วิธีใช้ชุดเรียน

อ่านแนวคิด → ทดลองกราฟ → ตอบคำถามพร้อมเหตุผล → ทำ Quiz และงานปฏิบัติ HTML ใช้งานออฟไลน์ได้โดยไม่ติดตั้งไลบรารี คำตอบฝึกหัดอยู่ในหน้านี้และไม่ส่งขึ้นเซิร์ฟเวอร์ เมื่อปิดหรือโหลดหน้าใหม่ คำตอบจะเริ่มใหม่

สื่อเสริมจาก Notebook: การสร้าง features ของข้อมูลรายวัน การอ่าน coefficients และ Bootstrap จัดเป็นส่วนต่อยอด ไม่เพิ่มเข้าไปเป็น LO ทางการของบทโดยอัตโนมัติ

บท 07 • LO1–LO2

เลือกชนิดการเรียนรู้จาก “สิ่งที่มี” และ “สิ่งที่ต้องการ”

Machine Learning เป็นส่วนหนึ่งของ AI ที่เรียนรู้รูปแบบจากข้อมูล การได้ข้อมูลมากขึ้นไม่ได้รับประกันว่าโมเดลจะดีขึ้น หากข้อมูลผิดหรือไม่ตรงกับงาน

วิธี	ข้อมูลที่ใช้เรียนรู้	ตัวอย่างงาน
Supervised มีคำตอบเป้าหมาย	X คือข้อมูลนำเข้า และ y คือคำตอบในอดีต	ทำนายเวลาส่งพัสดุจากข้อมูลการส่งที่มีเวลาจริง
Unsupervised ค้นหาโครงสร้าง	ไม่มี label เป้าหมายสำหรับงานนั้น	แบ่งกลุ่มรูปแบบการซื้อ เพื่อสำรวจลักษณะลูกค้า
Reinforcement เรียนรู้จากผลของการกระทำ	สถานะ การกระทำ และรางวัลที่สะสมตามเวลา	ฝึกตัวแทนจัดเส้นทางในระบบจำลองจากรางวัลและต้นทุน

Regression กับ Classification ต่างกันที่เป้าหมาย

“ส่งถึงในกี่นาที?” เป็น **Regression** เพราะต้องการปริมาณตัวเลข ส่วน “จะส่งทันหรือไม่?” เป็น **Classification** เพราะต้องการประเภท แม้ใช้ข้อมูลนำเข้าชุดเดียวกัน ก็เป็นคนละงานได้

ตัวเลขที่เป็นรหัส เช่น ประเภทสินค้า 1, 2, 3 ไม่ได้ทำให้งานนั้นกลายเป็น Regression ส่วนจำนวนสินค้าเป็นตัวเลขเชิงปริมาณ แม้เป็นจำนวนเต็ม ก็อาจใช้แบบจำลองถดถอยหรือโมเดลสำหรับข้อมูลนับ โดยตรวจว่าค่าพยากรณ์เหมาะกับงานหรือไม่

กฎที่แน่นอนอาจไม่ต้องใช้ ML

ถ้าค่าส่งกำหนดด้วยสูตรน้ำหนักและระยะทางที่แน่นอน ให้คำนวณตามกฎ แต่ถ้าต้องประมาณเวลาส่งที่มีการจราจร สภาพอากาศ และความไม่แน่นอน ML อาจช่วยเรียนรู้จากประวัติได้

ลองคิด 1 / 16

ระบบต้องทำนายเวลาส่งพัสดุเป็นนาทีจากงานในอดีตที่มีเวลาส่งจริง เป็นงานแบบใด?

- ○ Supervised regression
- ○ Unsupervised clustering
- ○ Reinforcement learning

การแบ่งข้อมูลคือการซ้อมสถานการณ์ใช้งานจริง

ก่อนฝึกโมเดล ให้กำหนดเป้าหมาย หน่วยของความผิดพลาด และเวลาที่ข้อมูลแต่ละตัวจะพร้อมใช้งาน

1. **กำหนดปัญหา:** พยากรณ์ยอดสั่งอาหารของวันพรุ่งนี้ เพื่อวางแผนวัตถุดิบ
2. **สำรวจและแบ่งข้อมูล:** ตรวจค่าหาย แถวซ้ำ หน่วย และแยกชุดข้อมูลให้ตรงกับเวลาใช้งาน
3. **สร้าง baseline:** เช่น ใช้ค่าเฉลี่ยจากชุดฝึก เพื่อดูว่าโมเดลใหม่ช่วยได้จริงแค่ไหน
4. **ฝึกด้วย training:** เรียนรู้ coefficients รวมถึงค่าเติมข้อมูลและค่าปรับสเกลจากชุดฝึกเท่านั้น
5. **เลือกด้วย validation:** เปรียบเทียบ features, degree หรือ alpha ภายใต้เกณฑ์เดียวกัน
6. **ประเมินด้วย test:** เมื่อเลือกวิธีแล้วจึงใช้ชุดที่กันไว้รายงานผลครั้งสุดท้าย ก่อนติดตามผลหลังใช้งาน

Training	Validation / CV	Test
เรียนรู้พารามิเตอร์	เลือกวิธีและความซับซ้อน	ตรวจผลหลังตัดสินใจ

หลีกเลี่ยง Data Leakage

ถ้าจะพยากรณ์ยอดของวันพรุ่งนี้ ห้ามใช้ "ยอดเงินจริงที่ปิดร้านพรุ่งนี้" เป็น feature เพราะยังไม่รู้ในเวลาพยากรณ์ ข้อมูลอากาศควรเป็นพยากรณ์อากาศที่มี ณ เวลานั้น ไม่ใช่อากาศจริงที่ทราบภายหลัง

ข้อมูลตามเวลาให้ฝึกจากอดีตและทดสอบกับช่วงถัดไป ส่วนข้อมูลหลายแถวต่อคนหรือร้าน ต้องระวังหน่วยเดียวกันปะปนข้ามชุด แบ่งแบบสุ่มได้เมื่อสอดคล้องกับสมมติฐานการใช้งาน

> **คำว่า test ในตัวอย่างเพื่อเรียน**
>
> หากเปิดดู error แล้วใช้ผลนั้นเลือก degree ซ้ำ ๆ ชุดนั้นกำลังทำหน้าที่เป็น validation แล้ว ควรมี final test อีกชุดที่ไม่ใช้ตัดสินใจ

ลองคิด 2 / 16

ทีมดู error ของชุดหนึ่งทุกครั้งเพื่อเลือก degree ชุดนี้ทำหน้าที่ใด?

- ○ Training
- ○ Validation
- ○ Final test ที่ยังเป็นอิสระจากการเลือก

เรียนรู้น้อยไป หรือจำข้อมูลฝึกมากไป?

Bias และ variance อธิบายความผิดพลาดจากคนละมุม ไม่ใช่ชื่อเรียก training error และ test error โดยตรง

Bias • คลาดเคลื่อนอย่างเป็นระบบ

ลองฝึกวิธีเดิมจากข้อมูลหลายชุด แล้วเฉลี่ยคำทำนาย ณ x เดิม ถ้าค่าเฉลี่ยยังห่างจากความสัมพันธ์จริง แสดงว่ามี bias ตัวอย่างเช่น ใช้เส้นตรงอธิบายแนวโน้มโค้งมาก ๆ

Variance • ไวต่อชุดฝึก

ใช้วิธีเดิมแต่เปลี่ยนชุดฝึก ถ้าคำทำนาย ณ x เดิมเปลี่ยนมาก แสดงว่ามี variance สูง ไม่ใช่เพียงข้อมูล y กระจายกว้าง

> $$\text{Expected squared prediction error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

แนวคิดสำหรับ squared loss ภายใต้ y = f(x) + noise ที่มีค่าเฉลี่ยศูนย์ และการเฉลี่ยเหนือชุดฝึก/ข้อมูลใหม่ Noise ที่ลดไม่ได้หมายถึงภายใต้ข้อมูลและกรอบปัญหาที่มีอยู่ ไม่ได้แปลว่าเก็บ feature เพิ่มแล้วจะช่วยไม่ได้เสมอ

สิ่งที่สังเกต	ข้อสงสัย	ลองตรวจหรือแก้
Train และ validation error สูงเทียบ baseline/เกณฑ์งาน	อาจ underfit	ตรวจข้อมูล เพิ่ม feature ที่เกี่ยวข้อง หรือเพิ่มความยืดหยุ่น
Train ต่ำ แต่ validation สูงกว่ามาก	อาจ overfit	ลดความซับซ้อน ใช้ regularization หรือเพิ่มข้อมูลที่ตรงงาน
ทั้งสองต่ำและใกล้กัน	เป็นสัญญาณที่ดี	ยังต้องตรวจ leakage และ final test

ช่องว่าง error เป็นสัญญาณให้ตรวจ ไม่ใช่หลักฐานพิสูจน์ bias/variance จากการแบ่งข้อมูลเพียงครั้งเดียว กราฟรูปตัว U เป็นภาพอธิบายที่พบบ่อย ไม่ใช่กฎว่าทุกโมเดลและทุกชุดข้อมูลต้องเป็นรูปนี้

ลองคิด 3 / 16

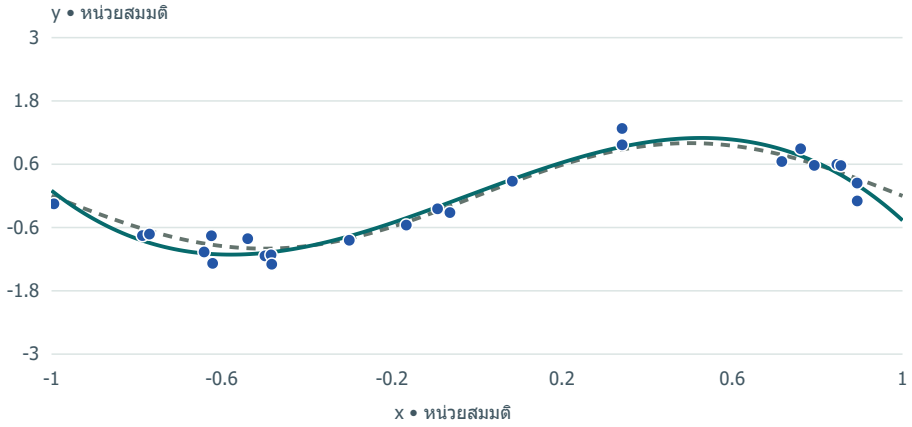
Train error ต่ำมาก แต่ validation error สูงกว่ามาก ควรตรวจเรื่องใดก่อน?

- ○ อาจ overfit หรือข้อมูลสองชุดมีความแตกต่างกัน
- ○ โมเดลถูกต้องแน่นอน
- ○ เพิ่ม degree เสมอ

ทดลอง • การเรียนรู้จากข้อมูลจำลอง

เปลี่ยน degree แล้วดูทั้งเส้นและความผิดพลาด

แบบทดลองนี้ fit polynomial จริงกับข้อมูลจำลอง 24 จุด และคำนวณ MSE กับ validation 80 จุดที่คงเดิม ข้อมูลใช้หน่วยสมมติ



ชุดฝึก 1/5 (seed 12) • degree 3 • Train MSE 0.0329 • Validation MSE 0.0503

วงกลม = ข้อมูลฝึก • เส้นทึบ = โมเดล • เส้นประ = ความสัมพันธ์จริงที่รู้ได้เพราะเป็นข้อมูลจำลอง กราฟตรึงแกน y ที่ −3 ถึง 3; ค่าที่เกินขอบไม่แสดงในกราฟ แต่รวมใน MSE ทั้งหมด

ลองตอบจากสิ่งที่เห็น

1. เมื่อเพิ่ม degree ค่า training MSE เปลี่ยนอย่างไร?
2. degree ที่ training MSE ต่ำที่สุด เป็นตัวที่ validation MSE ต่ำที่สุดด้วยหรือไม่?
3. คง degree เดิมแล้วเปลี่ยนชุดฝึก เส้นพยากรณ์เปลี่ยนมากแค่ไหน?

วิธีอ่านอย่างถูกต้อง

ปุ่มเปลี่ยนชุดฝึกวนผ่านข้อมูล 5 ชุดจาก seed ที่กำหนดไว้ ผลซ้ำเดิมได้ การดูเส้นหลายชุดช่วยให้เห็นความไวต่อชุดฝึก แต่ยังไม่ใช่การประมาณ bias² และ variance อย่างแม่นยำ ในงานจริงเราไม่รู้เส้นความสัมพันธ์จริงนี้

PDF แสดงสถานะเริ่มต้น degree 3 ดูตารางทุก degree และการเปลี่ยนเส้นแบบโต้ตอบได้ใน study_guide.html

ลองคิด 4 / 16

คง degree เดิม แล้วฝึกจากหลายชุด คำทำนายที่ x เดิมเปลี่ยนมาก สะท้อนแนวคิดใด?

- ○ Variance ของวิธีเรียนรู้
- ○ ค่าเฉลี่ยของ y
- ○ หน่วยของ RMSE

บท 08 • LO1

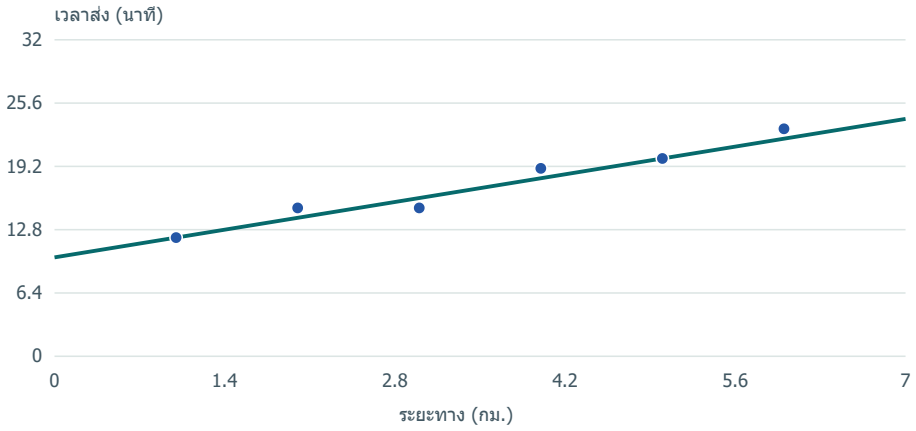
Linear Regression: อ่านเส้นให้เป็นความหมาย

โมเดลเส้นตรงใช้ค่าฐานและผลรวมของ features เพื่อประมาณตัวเลขเป้าหมาย

$$\hat{y} = b_0 + b_1 x \quad \text{หรือ} \quad \hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \dots$$

ŷ คือค่าพยากรณ์ **b₀** คือ intercept และ **b₁** คือ slope หรือ coefficient ของ x วิธี Ordinary Least Squares (OLS) เลือก coefficients ให้ผลรวม residual ยกกำลังสองในชุดฝึกต่ำที่สุด ภายใต้รูปแบบโมเดลที่กำหนด

ตัวอย่างจำลอง: **เวลาส่ง (นาที) = 10 + 2 × ระยะทาง (กม.)** ระยะ 5 กม. ได้คำทำนาย 20 นาที ความชัน 2 หมายถึงเมื่อระยะเพิ่ม 1 กม. ค่าพยากรณ์เพิ่ม 2 นาที ส่วนค่าฐาน 10 คือค่าที่โมเดลให้เมื่อระยะเป็นศูนย์ ซึ่งอาจอยู่นอกขอบเขตที่ข้อมูลรองรับ



ŷ = 10.00 + 2.00x • Training MSE = 0.667 นาที² • ระยะ 5 กม. → 20.00 นาที

จุด = เวลาจริงจำลอง • เส้นทึบ = ค่าพยากรณ์ • เส้นตั้งประ = residual หรือ y − ŷ ค่าบวกหมายถึงโมเดลทำนายต่ำกว่าค่าจริง แกนตั้งขยายเมื่อเส้นสูงเกินช่วงเริ่มต้น

ลองเลื่อนเส้นเพื่อให้ความผิดพลาดเล็กลง แล้วกด “ใช้เส้น OLS” เพื่อเปรียบเทียบ การผ่านทุกจุดไม่ใช่เป้าหมายเมื่อข้อมูลมี noise

ลองคิด 5 / 16

ใน ŷ = 10 + 2x เมื่อ x = 5 กม. ค่าพยากรณ์คือเท่าใด?

- ○ 12 นาที
- ○ 20 นาที
- ○ 25 นาที

บท 08 • LO3

เลือกตัวชี้วัดให้ตรงกับต้นทุนของการทำนายผิด

รายงานคะแนนพร้อมหน่วย ชุดข้อมูลที่ใช้ และ baseline เสมอ ค่าเดียวไม่เล่าทุกด้านของโมเดล

ตัวชี้วัด	คำนวณจาก residual $e = y - \hat{y}$	ความหมาย
MAE	ค่าเฉลี่ย $\lvert e \rvert$	พลาดสัมบูรณ์โดยเฉลี่ย หน่วยเดียวกับ y
MSE	ค่าเฉลี่ย e^2	เน้นข้อผิดพลาดใหญ่ หน่วยของ y ยกกำลังสอง
RMSE	√MSE	เน้นข้อผิดพลาดใหญ่ แต่กลับมาใช้หน่วยเดียวกับ y
R^2	$1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$	เทียบกับการใช้ค่าเฉลี่ย y ของชุดที่ประเมิน ไม่มีหน่วย

ตัวอย่างคำนวณครบหนึ่งชุด

เวลาจริง = [10, 20, 30] นาที; ค่าพยากรณ์ = [12, 18, 26] นาที

residual = [−2, 2, 4] → $\lvert e \rvert$ รวม 8 → e^2 รวม 24

2.67	8.00	2.83	0.88
MAE • นาที	MSE • นาที²	RMSE • นาที	R^2 • ไม่มีหน่วย

$R^2 = 1 - 24/200 = 0.88$ ไม่ได้หมายความว่า "ทำนายถูก 88%" และ R^2 อาจติดลบเมื่อพลาดมากกว่าการใช้ค่าเฉลี่ยของชุดประเมิน หาก y คงที่ ตัวหารเป็นศูนย์ ต้องตรวจวิธีจัดการของซอฟต์แวร์

อ่านคะแนนเป็นภาษาของผู้ใช้งาน

"ในข้อมูลวันถัดมาที่กันไว้ โมเดลพลาดเฉลี่ย 2.67 นาที และ RMSE 2.83 นาที" ชัดกว่าพูดว่า "โมเดลดีมาก" หากพลาดช้าเกินกำหนดแพงกว่าพลาดเร็ว ควรตรวจความผิดพลาดสองทิศทางแยกกัน เพราะ MAE/RMSE ให้โทษตามขนาด ไม่แยกต้นทุนของทิศทาง

การเทียบ R^2 กับค่าเฉลี่ยของชุดประเมินเป็นนิยามของ metric ส่วน baseline ที่ใช้พยากรณ์จริงต้องเรียนค่าจาก training เท่านั้น

ลองคิด 6 / 16

ถ้า MSE = 9 นาที² ค่า RMSE เท่ากับเท่าใด?

- ○ 3 นาที
- ○ 9 นาที
- ○ 81 นาที²

บท 07 • LO5 / บท 08 • LO2

fit → predict → ประเมินผล: รู้ว่าข้อมูลไปที่ไหน

ตัวอย่างนี้สมมติว่าตารางแบ่งตามเวลาไว้แล้ว ทุก feature พร้อมใช้งาน ณ เวลาพยากรณ์ และไม่มีค่าหาย

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

# ตาราง train, validation, test แยกไว้ก่อนแล้ว
features = ["distance_km", "orders_in_queue"]
X_train = train[features]          # ตารางสองมิติ
y_train = train["delivery_min"]   # คำตอบเป้าหมาย

model = LinearRegression()
model.fit(X_train, y_train)       # เรียน coefficients
pred = model.predict(validation[features])
mae = mean_absolute_error(validation["delivery_min"], pred)
rmse = mean_squared_error(validation["delivery_min"], pred) ** 0.5
r2 = model.score(validation[features], validation["delivery_min"])
print(mae, rmse, r2)
```

`fit` เรียนรู้จากข้อมูลฝึก `predict` คืนคำทำนาย `score` คืนคะแนนตามชนิด estimator: สำหรับ `LinearRegression` คือ R^2 ไม่ใช่ accuracy เหมือนกันทุกโมเดล

Simple กับ Multiple

ใช้เพียง `["distance_km"]` เป็น Simple Linear Regression ใช้ระยะทางและคิวงานร่วมกันเป็น Multiple Linear Regression ให้ประเมินบนชุดเดียวกันเพื่อเทียบอย่างยุติธรรม

หากสมการคือ $\hat{y} = 8 + 2\times$ระยะทาง $+ 1.5\times$คิวงาน coefficient ของระยะทางหมายถึงค่าพยากรณ์เพิ่ม 2 นาทีต่อ 1 กม. **เมื่อ features อื่นคงเดิม** ความสัมพันธ์นี้ยังไม่พิสูจน์เหตุและผล และห้ามเทียบขนาด coefficient ข้ามหน่วยอย่างตรง ๆ เพื่อจัดอันดับความสำคัญ

Polynomial เช่น x และ x^2 ยังเป็นโมเดลเชิงเส้นใน coefficients แม้เส้นพยากรณ์จะโค้ง การเพิ่ม degree ทำให้ยืดหยุ่นขึ้นและอาจ overfit ได้

ลองคิด 7 / 16

LinearRegression.score(X, y) คืนคะแนนใด?

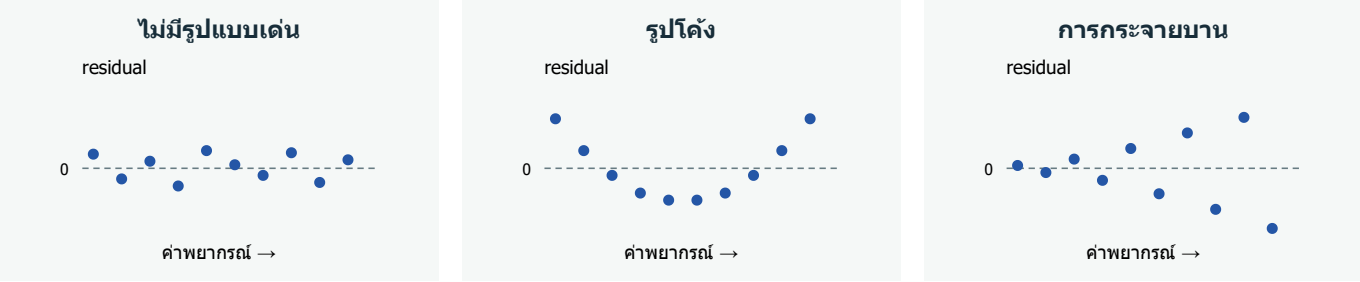
- ○ Accuracy เสมอ
- ○ MAE
- ○ R^2

บท 08 • LO4–LO5

ดูว่าผิดตรงไหน แล้วแก้ให้ตรงสาเหตุ

Residual plot ช่วยตั้งข้อสงสัยเกี่ยวกับโมเดล ต้องอ่านร่วมกับบริบท ไม่ใช่ใช้กราฟเดียวประกาศว่าข้อสมมติผ่านทั้งหมด

ลักษณะ residual	สิ่งที่ควรตรวจ	แนวทางทดลอง
กระจายรอบศูนย์ ไม่มีรูปแบบชัด	ไม่มีสัญญาณปัญหาเด่นในกราฟนี้	ตรวจตามเวลา กลุ่มข้อมูล และข้อผิดพลาดใหญ่เพิ่ม
เป็นแนวโค้ง	รูปแบบความสัมพันธ์อาจยังไม่ครบ	ลอง polynomial / interaction ที่มีเหตุผล
บานเป็นรูปกรวย	ความแปรปรวนของ error อาจไม่คงที่	ตรวจการแปลงเป้าหมาย/แบบจำลองความไม่แน่นอน
บวกหรือลบต่อกันเป็นช่วงเวลา	อาจมี seasonality หรือความสัมพันธ์ตามเวลา	เพิ่ม feature ที่พร้อมใช้ และประเมินด้วย time split
จุดโดดห่างจากกลุ่มมาก	ข้อมูลผิด หรือเหตุการณ์จริงที่ต่างออกไป?	ตรวจต้นทางและอิทธิพลก่อนตัดข้อมูล



ภาพสาธิตรูปแบบเพื่อฝึกอ่าน แกนนอน = ค่าพยากรณ์ แกนตั้ง = residual ไม่ใช่ผลวิเคราะห์จากข้อมูลจริง

ความปกติของ residual เกี่ยวข้องกับการอนุมาน/ช่วงความเชื่อมั่นแบบคลาสสิก ไม่ใช่เงื่อนไขว่าต้องปกติเสมอจึงคำนวณ OLS หรือพยากรณ์ได้

ลองคิด 8 / 16

residual มีรูปโค้งอย่างเป็นระบบ ข้อใดเป็นแนวทางทดลองที่มีเหตุผล?

- ○ ลองเพิ่มเทอมโค้งที่เกี่ยวข้อง แล้วเทียบบน validation
- ○ ลบทุกจุดที่ residual ไม่เป็นศูนย์
- ○ ใช้ test เลือก degree ไปเรื่อย ๆ

บท 08 • LO5

ปรับโมเดลทีละเหตุผล แล้ววัดผลบนข้อมูลเดิม

เริ่มจากข้อผิดพลาดที่พบ กำหนดสิ่งที่จะเปลี่ยน และเปรียบเทียบกับ baseline ภายใต้การแบ่งข้อมูลเดียวกัน

Regularization ลดการพึ่งพา coefficients ขนาดใหญ่

Ridge (L2) เพิ่มโทษตามผลรวม coefficient² จึงหด coefficients ส่วน **Lasso (L1)** เพิ่มโทษตามผลรวมค่าสัมบูรณ์ และอาจทำให้บาง coefficient เป็นศูนย์ เมื่อ alpha มากขึ้น การหดโดยทั่วไปแรงขึ้น แต่ไม่รับประกันว่าคะแนนข้อมูลใหม่จะดีขึ้น

ควรปรับสเกล features ก่อนใช้ penalty และ fit ตัวปรับสเกลจาก training เท่านั้น ใช้ pipeline เพื่อให้ preprocessing อยู่ในขั้นตอนฝึก เลือก alpha ด้วย validation/CV ไม่ใช่ final test

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

ridge = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
ridge.fit(X_train, y_train)
pred = ridge.predict(validation[features])
# เปรียบเทียบ MAE/RMSE กับ LinearRegression บน validation เดิม
```

Feature Selection: เลือกตัวแปรที่มีเหตุผลและมีจริง ณ เวลาพยากรณ์ ทดสอบบน validation การตัด feature จนคะแนนดีบน test ที่เปิดดูซ้ำเป็นการปรับให้เข้ากับ test เช่นกัน หาก features สัมพันธ์กันมาก coefficients อาจไม่เสถียร

ตัวอย่างวงรอบการปรับปรุง

1. ถ้า residual โค้ง ลองเพิ่มเทอมกำลังสองที่มีเหตุผล
2. ถ้า train ดีมากแต่ validation แย่ ลองลด degree หรือเพิ่ม penalty
3. ถ้า feature ใหม่ทำให้ validation ดีขึ้น ตรวจว่าข้อมูลนั้นมีอยู่จริงตอนพยากรณ์
4. บันทึกสิ่งที่ลองและผล ไม่เลือกเฉพาะการแบ่งข้อมูลที่ให้คะแนนดี
5. เมื่อเลือกวิธีเสร็จ จึงประเมิน final test และรายงานข้อจำกัด

ประยุกต์ • จากโมเดลสู่การตัดสินใจ

ตัวอย่างงานจริง: วางแผนรอบส่งและจำนวนรถ

สถานการณ์ออกแบบเพื่อการเรียน: ผู้จัดการต้องรู้เวลาส่งโดยประมาณก่อนรับงาน ไม่ใช่หลังส่งเสร็จ ตัวเลขต่อไปนี้เป็นตัวอย่างจำลอง

กำหนดงานให้ตรวจสอบได้

- หนึ่งแถว = หนึ่งงานส่ง
- Target = เวลาตั้งแต่รับงานถึงส่งเสร็จ หน่วย นาที
- Features = ระยะทาง คิวงาน เวลาเริ่ม และสภาพจราจรที่ทราบตอนรับงาน
- แบ่งตามเวลาและแยกข้อมูลซ้ำ

เกณฑ์เลือกโมเดล

- เริ่มจาก baseline ของ training
- รายงาน MAE/RMSE บน validation
- ตรวจช่วงเร่งด่วนและระยะทางไกลแยก
- ตรวจความเสี่ยงจากการประเมินเวลาต่ำเกินไป

วิธี • ค่าจำลอง	Train MAE	Validation MAE
Baseline	8 นาที	9 นาที
Linear Regression	5 นาที	6 นาที
Polynomial ซับซ้อน	1 นาที	11 นาที

ถ้าใช้ validation MAE เป็นเกณฑ์ และปัจจัยอื่นเท่ากัน เลือก Linear Regression เพื่อประเมินต่อด้วย final test แม้ polynomial จะพลาดในชุดฝึกน้อยกว่า ข้อมูลนี้ยังไม่ใช่หลักฐานว่าใช้งานจริงแล้วคุ้มค่า

สิ่งที่ส่งมอบให้ทีมงาน

รายงานช่วงเวลาข้อมูล รุ่นโมเดล features ที่ใช้ คะแนนพร้อม baseline และกรณีที่พลาดมาก อธิบายว่าเป็นค่าประมาณ ไม่ใช่เวลารับประกัน หลังใช้งานติดตาม error เมื่อมีค่าจริง และตรวจว่ารูปแบบข้อมูลเปลี่ยนหรือไม่

LLM ช่วยได้อย่างไร? • บท 07 LO6

ใช้ช่วยร่างโค้ด อธิบาย error และร่างคำอธิบายผล โดยให้โครงสร้างตารางหรือข้อมูลจำลองแทนข้อมูลส่วนบุคคล ตรวจชื่อ API กับเอกสาร รันโค้ดเอง ตรวจ leakage และคำนวณตัวอย่างเล็กด้วยมือก่อนเชื่อผล ไม่ให้ LLM แต่งคะแนนทดลองหรือสรุปว่าเกิดเหตุและผลจาก coefficient

ลองคิด 9 / 16

LLM สร้างโค้ดได้โดยไม่มี error ข้อใดควรทำต่อ?

- ○ รายงานว่าผลถูกต้องทันที
- ○ ตรวจการแบ่งข้อมูล สูตร และผลกับตัวอย่างที่รู้คำตอบ
- ○ ให้ LLM สร้างคะแนนผลทดลองแทนการรัน

ต่อยอดจาก NOTEBOOK • ไม่ใช่ LO ใหม่ของบท

ข้อมูลรายวันและ Bootstrap: เล่าให้ครบทั้งผลและความไม่แน่นอน

Notebook ที่ให้มาสาธิตจำนวนนักปั่นรายวัน โดยสร้าง features จากวัน สภาพอากาศ และแนวโน้มเวลา แนวคิดนี้ประยุกต์กับความต้องการใช้บริการรายวันได้

Feature Engineering ที่คำนึงถึงเวลาพยากรณ์

วันในสัปดาห์และวันหยุดเป็นข้อมูลที่รู้ล่วงหน้า ส่วนอุณหภูมิหรือฝนจริงทั้งวันยังไม่รู้ตอนเริ่มวัน จึงต้องแยกงาน "อธิบายข้อมูลอดีต" ออกจากงาน "พยากรณ์วันพรุ่งนี้" หากพยากรณ์ ให้ใช้ข้อมูลที่มีจริงในเวลานั้น

ถ้าใช้ dummy ของวันครบ 7 วัน แต่ละวันมีค่า 1 เพียงคอลัมน์เดียว การใช้ `fit_intercept=False` ทำให้แต่ละวันมีค่าฐานของตนเอง อีกวิธีคือใช้ intercept และตัด dummy หนึ่งวันเป็นวันอ้างอิง หลีกเลี่ยงการมีทั้ง intercept กับ dummy ครบทุกกลุ่ม เมื่อจะตีความ coefficients

Bootstrap ทำอะไร?

1. สำหรับแถวที่เป็นอิสระ สุ่มแถวจาก training แบบใส่คืนให้ได้จำนวนเท่าเดิม
2. ฝึกโมเดลใหม่ในแต่ละรอบ และเก็บ coefficient ที่สนใจ
3. ดูการกระจายหรือส่วนเบี่ยงเบนมาตรฐานของ coefficient เพื่ออธิบายความไม่แน่นอนจากการสุ่มตัวอย่าง

ค่า std จาก bootstrap ไม่ใช่ขอบเขตความผิดพลาดของคำทำนายแต่ละวันโดยตรง ข้อมูลรายวันที่พึ่งพากันอาจต้องใช้ block bootstrap หรือวิธีที่คงโครงสร้างเวลาไว้ การสุ่มแต่ละวันแบบอิสระอาจให้ภาพความไม่แน่นอนที่ไม่เหมาะสม

> **อ่านผลเชิงสัมพันธ์อย่างระมัดระวัง**
>
> coefficient ของอุณหภูมิเป็นบวก หมายถึงโมเดลให้ค่าพยากรณ์สูงขึ้นเมื่ออุณหภูมิเพิ่ม โดยคง features อื่นไว้ ไม่ใช่หลักฐานว่าอุณหภูมิเป็นสาเหตุเพียงอย่างเดียว หรือพิสูจน์ผลของการลงทุนโครงสร้างพื้นฐาน

ลองคิด 10 / 16

ใน Multiple Linear Regression จะตีความ coefficient ของระยะทางอย่างไร?

- ○ ผลต่อค่าพยากรณ์เมื่อระยะทางเพิ่มหนึ่งหน่วย โดยคง features อื่น
- ○ ผลเชิงสาเหตุที่พิสูจน์แล้ว
- ○ ความน่าจะเป็นที่คำทำนายถูก

ลองคิด 11 / 16

ข้อมูลรายวันมีความสัมพันธ์ตามเวลา ข้อใดเหมาะกับการประเมินความไม่แน่นอนมากกว่า?

- ○ สุ่มแต่ละวันแบบอิสระโดยไม่ตรวจโครงสร้าง
- ○ พิจารณา block bootstrap ที่รักษาช่วงเวลาต่อเนื่อง
- ○ สรุปว่าค่า std ของ coefficient คือ MAE

ทบทวน • นำความรู้ไปใช้

ก่อนบอกว่าโมเดลนี้พร้อม ลองอธิบาย 6 ข้อนี้

ถ้าอธิบายด้วยคำของตนเองและแสดงผลจากโค้ดได้ จะเป็นหลักฐานการเรียนรู้ที่ชัดกว่าจำชื่ออัลกอริทึม

1. งานนี้ต้องการตัวเลข ประเภท กลุ่ม หรือการตัดสินใจตามรางวัล?
2. รู้ feature แต่ละตัวในเวลาที่ต้องพยากรณ์จริงหรือยัง?
3. ใช้ชุดไหนฝึก ชุดไหนเลือก และชุดไหนประเมินครั้งสุดท้าย?
4. error ที่รายงานมีหน่วยอะไร และดีกว่า baseline เท่าใด?
5. residual ชี้ให้ตรวจอะไรเพิ่ม และจะแก้โดยไม่ใช้ test ซ้ำอย่างไร?
6. ข้อสรุปไหนเป็นสิ่งที่ข้อมูลแสดง และข้อไหนยังพิสูจน์ไม่ได้?

ลองคิด 12 / 16

โมเดล A มี validation MAE 4 นาที และ B มี 7 นาที บนชุดเดียวกัน ถ้าใช้ MAE เป็นเกณฑ์และปัจจัยอื่นเท่ากัน ควรเลือกตัวใด?

- ○ A แล้วประเมินด้วย final test
- ○ B เพราะ error มากกว่า
- ○ ตัดสินจากตัวเลขนี้ไม่ได้เลย

แหล่งอ่านประกอบ

1. บทที่ 7: แนะนำ Machine Learning — แหล่ง LO และเนื้อหาหลัก
2. บทที่ 8: Regression — แหล่ง LO และเนื้อหาหลัก
3. MLU-Explain: Bias–Variance Tradeoff — ภาพอธิบายการเปลี่ยนชุดฝึกและความซับซ้อน
4. MLU-Explain: Linear Regression — ภาพอธิบายเส้นและความคลาดเคลื่อน
5. Notebook ต้นฉบับในโฟลเดอร์: Lecture_08_Bias_vs_Variance.ipynb และ Copy_of_Lecture_09_Linear_Regression_TH (1).ipynb / (2).ipynb ซึ่งเป็นสำเนาเดียวกัน รวมถึง Copy_of_Lecture_09_Regression_Student_Exercise.ipynb (ส่วนเพิ่มเติมท้ายบททบทวน)

จัดทำ 22 กันยายน 2026 • เว็บใช้รหัส CP352101 แต่ Notebook ใช้ SC312104 จึงอ้างตามชื่อเนื้อหาและแยกแหล่ง LO ไว้ ตัวอย่างและคำถามในชุดนี้เขียนขึ้นใหม่ ไม่ใช่ผลทดลองจากชั้นเรียนหรือข้อมูลกิจการจริง

เพิ่มเติมจาก STUDENT EXERCISE • EXT-EX-1–3

สร้างข้อมูลที่รู้คำตอบ เพื่อเข้าใจว่าโมเดลเรียนรู้อะไร

เมื่อเรากำหนดสมการสร้างข้อมูลเอง จะตรวจได้ว่าการ fit กู้ความสัมพันธ์กลับมาได้ใกล้เคียงเพียงใด โดยไม่คาดหวังว่าตัวอย่างที่มี noise ต้องได้ coefficient ตรงเป๊ะ

$$y = ax + b + \varepsilon \quad \text{โดย } \varepsilon \text{ มีค่าเฉลี่ย } 0$$

```
import numpy as np
rng = np.random.default_rng(123)
x = rng.uniform(0, 10, size=50)
noise = rng.normal(loc=0, scale=1, size=50)
y = 2 * x - 3 + noise       # ความชัน 2; intercept -3
X = x.reshape(-1, 1)         # 50 แถว, 1 feature
```

สัญญาณ $2x - 3$ เขียนในรูป $ax + b$ ได้โดย $a = 2$ และ $b = -3$ ถ้าเปลี่ยนค่าเฉลี่ย noise เป็น 0.5 ค่าเฉลี่ยของ y เมื่อรู้ x จะเป็น $2x - 2.5$ จึงต้องแยก “ค่าฐานในโค้ด” ออกจาก “ค่าฐานของความสัมพันธ์เฉลี่ย”

Seed ทำให้สร้างลำดับสุ่มซ้ำได้เมื่อใช้ generator วิธีและลำดับคำสั่งเดิม ไม่ทำให้ข้อมูลไร้ noise หรือทำให้โมเดลแม่นขึ้น อย่าเขียน `np.random.seed = 123` เพราะเป็นการเขียนทับชื่อฟังก์ชัน ใช้ generator ในตัวอย่างข้างบนแทน

Cost function: ให้คะแนนเส้นแต่ละเส้น

SSE = Σ(y − ŷ)² เป็นผลรวม ส่วน **MSE = SSE/n** เป็นค่าเฉลี่ย บนข้อมูลชุดเดียวกันที่ n คงที่และยังไม่มี penalty ทั้งสองเลือกเส้นที่ดีที่สุดตรงกัน แต่เมื่อนำ penalty มารวม ต้องระบุการหารด้วย n ด้วย เพราะทำให้ความหมายของ alpha ต่างกัน

ลองใช้กราฟเส้นตรงในบทก่อนหน้า: การเลื่อน slope/intercept คือการเดินบนพื้นผิว cost ในแกนพารามิเตอร์ สำหรับ OLS ที่กำหนด features คงที่ พื้นผิว squared-error เป็น convex ค่าต่ำสุดเฉพาะที่จึงเป็นค่าต่ำสุดโดยรวม แต่คำตอบอาจไม่เป็นเอกลักษณ์ถ้า design matrix มีคอลัมน์ที่พึ่งพากัน

ลองคิด 13 / 16

ใน y = 2x − 3 + noise เมื่อ noise มีค่าเฉลี่ยศูนย์ intercept ของความสัมพันธ์เฉลี่ยคือเท่าใด?

- ○ 3
- ○ −3
- ○ 0

ลองคิด 14 / 16

บนข้อมูลชุดเดียวกัน 50 แถว ไม่มี penalty ถ้า SSE = 100 จะได้ MSE เท่าใด?

- ○ 2
- ○ 50
- ○ 100

อ้างอิง Student Exercise เซลล์ 0–18 และ 25–38 • เรียบเรียงและเขียนตัวอย่างใหม่โดยใช้ noise ค่าเฉลี่ยศูนย์

เพิ่มเติมจาก STUDENT EXERCISE • EXT-EX-4

เปลี่ยน features ให้เส้นตรงอธิบายความสัมพันธ์โค้งได้

Basis function คือฟังก์ชันที่ใช้สร้าง features ใหม่จาก x เดิม จากนั้น Linear Regression เรียนรู้น้ำหนักของแต่ละ feature

Polynomial basis

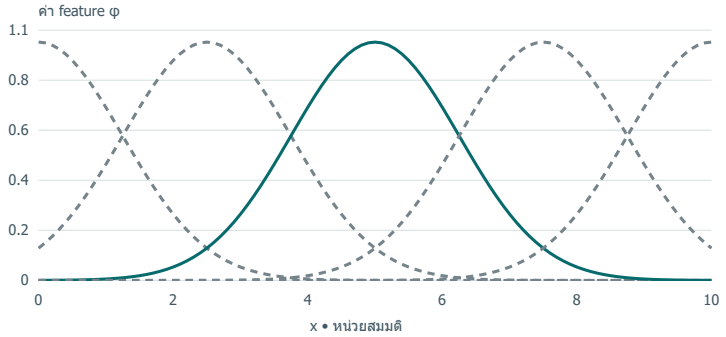
เปลี่ยน x เป็น $[x, x^2, x^3]$ เช่น x = 2 ได้ [2, 4, 8] ใช้ `PolynomialFeatures(3, include_bias=False)` แล้วให้โมเดลมี intercept ของตนเอง

Gaussian basis

แต่ละ feature ให้ค่าสูงเมื่อ x ใกล้จุดศูนย์กลาง c_j และลดลงเมื่อห่างออกไป ความกว้าง σ ควบคุมการลดลงนี้ ค่า feature ไม่ใช่ความน่าจะเป็น

$$\varphi_j(x) = \exp[-\tfrac{1}{2}((x - c_j)/\sigma)^2]$$
$$\hat{y} = b_0 + b_1\varphi_1(x) + \ldots + b_n\varphi_n(x)$$

เมื่อ centers และ widths ถูกกำหนดไว้ โมเดลยังเป็นเชิงเส้นใน coefficients แม้ไม่เป็นเส้นตรงใน x ตารางเดิมมีรูปร่าง (จำนวนแถว, 1) หลังแปลงด้วย N Gaussian bases จะเป็น (จำนวนแถว, N)



N = 5 • ระยะห่าง centers = 2.50 • σ = 1.25 • ที่ x = 5 ค่า basis กลาง = 1

เส้นทึบสีเขียว = basis ที่ center 5 • เส้นประสีเทา = bases อื่น แต่ละเส้นเป็น feature หนึ่งคอลัมน์ ยังไม่ใช่เส้นพยากรณ์ที่ผ่านการ fit ข้อมูล

ในคลาสของ Notebook: **σ = width_factor × ระยะห่างระหว่าง centers** ไม่ใช่ σ = width_factor โดยตรง แบบทดลองนี้วาง centers บนช่วงจำลอง 0–10 ถ้า N = 5 ระยะห่าง 2.5 และ factor = 0.5 จะได้ σ = 1.25 เมื่อเปลี่ยน N โดยคง factor ไว้ ความกว้างจริงก็เปลี่ยนด้วย

ลองคิด 15 / 16

Centers ห่างกัน 2.5 และ GaussianFeatures ใช้ width_factor = 0.5 ความกว้างจริง σ เป็นเท่าใด?

- ○ 0.5
- ○ 1.25
- ○ 2.5

อ้างอิง Student Exercise เซลล์ 40–49 • แบบทดลองนี้แสดงการแปลง features ไม่วัดประสิทธิภาพโมเดล

เพิ่มเติมจาก STUDENT EXERCISE • EXT-EX-5

เปรียบเทียบ regularization โดยคงสิ่งอื่นให้เท่ากัน

หากเปลี่ยนทั้งจำนวน basis และวิธีลงโทษพร้อมกัน จะบอกไม่ได้ว่าผลที่ต่างเกิดจากการเปลี่ยนส่วนใด

วิธี	สิ่งที่ลดให้ต่ำ	สิ่งที่สังเกต
OLS	Squared error	อาจมี coefficients ขนาดใหญ่ที่หักล้างกัน
Ridge	Squared error + L2 penalty	หด coefficients โดยทั่วไปไม่ทำให้เป็นศูนย์พอดี
Lasso	Squared error + L1 penalty	บาง coefficients อาจเป็นศูนย์

ลำดับการทดลองที่อ่านเหตุผลได้

1. แบ่ง training/validation/test แล้วตรึงการแบ่งเดิมสำหรับทุกวิธี
2. เลือกจำนวน basis, centers และ widths ชุดเดียวกัน เรียนส่วนที่ต้อง fit จาก training เท่านั้น
3. ถ้าปรับสเกล features ให้ fit scaler ใน training และใช้ขั้นตอนเดียวกันทุกวิธี
4. เปรียบเทียบ OLS, Ridge และ Lasso บน validation เดิม ปรับ alpha ภายในแต่ละวิธี ไม่ใช้เลข alpha เท่ากันเป็นหลักฐานว่า penalty แรงเท่ากัน
5. บันทึกทั้ง error และ coefficients แล้วประเมิน final test หลังเลือกวิธีแล้ว

ตัวอย่างสมมติ: OLS มี train MSE 0.01 และ validation MSE 0.80 ส่วน Ridge มี 0.04 และ 0.12 หากใช้ validation MSE เป็นเกณฑ์และเงื่อนไขอื่นเท่ากัน ให้เลือก Ridge แม้ training error สูงขึ้น ตัวเลขนี้เป็นตัวอย่างประกอบ ไม่ใช่ผลรัน Notebook

> **coefficients ใหญ่ไม่พิสูจน์ว่า overfit**
>
> ค่าขนาดใหญ่และสลับเครื่องหมายอาจเกิดจาก features ที่สัมพันธ์กัน หรือสเกลต่างกัน ต้องดูผลบนข้อมูลใหม่และความไวเมื่อเปลี่ยนชุดฝึกประกอบ การเพิ่ม penalty ไม่ได้รับประกันว่าจะดีขึ้นเสมอ

นำไปใช้กับงานจริงอย่างไร?

ถ้าความต้องการใช้บริการเพิ่มเฉพาะช่วงอุณหภูมิหนึ่ง Gaussian bases อาจอธิบายรูปแบบเฉพาะช่วงได้ดี แต่ควรเปรียบเทียบกับโมเดลง่าย และระวังการพยากรณ์นอกช่วง training เพราะ Gaussian features จะเข้าใกล้ศูนย์เมื่อห่างจากทุก center

ลองคิด 16 / 16

ต้องการตรวจผลของ penalty โดยตรง ควรเปรียบเทียบโมเดลอย่างไร?

- ○ เปลี่ยนจำนวน basis และชุด validation ของแต่ละวิธี
- ○ คง basis, preprocessing และชุดข้อมูลไว้ แล้วเปรียบเทียบ penalty
- ○ เลือกวิธีจาก coefficient ที่ดูเล็กที่สุดอย่างเดียว

อ้างอิง Student Exercise เซลล์ 51–61 • ส่วนนี้เพิ่มความชัดเจนเรื่องเงื่อนไขการเปรียบเทียบ

เฉลยสำหรับอ่านหลังลองทำ

คำตอบและเหตุผลของแบบฝึกหัด 1–8

1. **Supervised regression**
   มีคำตอบ y เป็นเวลาในอดีต และเป้าหมายเป็นปริมาณตัวเลข จึงเป็น supervised regression

2. **Validation**
   ข้อมูลที่ใช้ตัดสินใจเลือกโมเดลทำหน้าที่เป็น validation ควรกัน final test อีกชุด

3. **อาจ overfit หรือข้อมูลสองชุดมีความแตกต่างกัน**
   เป็นสัญญาณให้ตรวจ overfitting รวมถึงคุณภาพและความต่างของข้อมูล ไม่ใช่ข้อพิสูจน์จากคะแนนอย่างเดียว

4. **Variance ของวิธีเรียนรู้**
   Variance พิจารณาความไวของคำทำนายต่อการเปลี่ยนชุดฝึก โดยดูที่ input เดิม

5. **20 นาที**
   แทนค่า 10 + 2×5 = 20 นาที และต้องระบุหน่วยของผลลัพธ์

6. **3 นาที**
   RMSE เป็นรากที่สองของ MSE จึงได้ 3 และกลับมาใช้หน่วยนาที

7. **R^2**
   score ของ LinearRegression คือ R^2 ความหมายของ score ขึ้นอยู่กับ estimator

8. **ลองเพิ่มเทอมโค้งที่เกี่ยวข้อง แล้วเทียบบน validation**
   รูปโค้งชวนให้ตรวจรูปแบบความสัมพันธ์ที่โมเดลยังอธิบายไม่ครบ ต้องประเมินการเปลี่ยนบน validation

เฉลยสำหรับอ่านหลังลองทำ

คำตอบและเหตุผลของแบบฝึกหัด 9–16

9. **ตรวจการแบ่งข้อมูล สูตร และผลกับตัวอย่างที่รู้คำตอบ**
   โค้ดรันได้ยังอาจผิดหลักการ ต้องตรวจข้อมูล การแบ่งชุด สูตร และค่าจริงที่คำนวณได้

10. **ผลต่อค่าพยากรณ์เมื่อระยะทางเพิ่มหนึ่งหน่วย โดยคง features อื่น**
    เป็นความสัมพันธ์ในโมเดลภายใต้ features อื่นคงเดิม ไม่ใช่ผลเชิงสาเหตุโดยอัตโนมัติ

11. **พิจารณา block bootstrap ที่รักษาช่วงเวลาต่อเนื่อง**
    การสุ่มเป็นช่วงอาจรักษาความพึ่งพาตามเวลาได้เหมาะกว่า ต้องเลือกวิธีและขนาดช่วงตามข้อมูล

12. **A แล้วประเมินด้วย final test**
    A มี MAE ต่ำกว่าในเงื่อนไขที่กำหนด จึงเป็นตัวเลือกเพื่อทดสอบครั้งสุดท้าย ยังไม่ใช่ข้อรับประกันผลในอนาคต

13. **−3**
    เขียนเป็น ax + b ได้โดย a = 2 และ b = −3 จึงมี intercept −3

14. **2**
    MSE = SSE/n = 100/50 = 2 การหารด้วย n คงที่ไม่เปลี่ยนเส้นที่ทำให้ error ต่ำสุด

15. **1.25**
    คลาสใน Notebook คำนวณ σ = factor × ระยะห่าง centers จึงได้ 0.5×2.5 = 1.25

16. **คง basis, preprocessing และชุดข้อมูลไว้ แล้วเปรียบเทียบ penalty**
    คงเงื่อนไขอื่นเพื่อแยกผลของ penalty และใช้ validation ประเมินคุณภาพ ไม่ตัดสินจากขนาด coefficient เพียงอย่างเดียว